INT-AWARE

Intent-Aware Deception System for Real-Time Attacker Profiling

Project Status | Enhancement Backlog | Novelty Analysis

Field	Value
Report date	11 September 2026
Author	Rakshita Shetty
Repository branch	remediation/int-aware (base: master)
Source of truth	PROJECT_AUDIT.md, ROADMAP.md, README.md, docs/evaluation/*, git log
Stack	Cowrie honeypot -> FastAPI + calibrated ML + optional Gemini -> MongoDB -> React
Test suite	82 test functions across 18 modules (hermetic; 74 at audit baseline)
Honest ML accuracy	94.49% same-template held-out / 67.87% leave-families-out (generalisation)

About this document

This report summarises (1) what has actually been built and verified in INT-AWARE, (2) the enhancement backlog you can pull from next, and (3) a candid analysis of where the genuine, defensible novelty of the project lies. In keeping with the project's own anti-fabrication rule, every figure is quoted from committed artifacts and each proposed feature is clearly labelled as **done**, **in progress**, or **proposed**.

1. Executive Summary

INT-AWARE is a full-stack cyber-deception platform. A real Cowrie SSH/Telnet honeypot (emulated shell — nothing ever executes on the host) writes JSON event logs; a FastAPI backend tails those logs in real time, classifies each command's **attacker intent** across a 10-class MITRE-aligned taxonomy, scores risk on a deterministic 0–100 scale, aggregates per-session and per-attacker intelligence into MongoDB, and streams everything to a React analyst dashboard over REST and WebSockets.

The engineering core is real and, in several places, genuinely well-built: the honeypot is safely isolated, the log tailer is rotation- and truncation-aware, the classifier arbitration is coherent, the risk engine is additive and explainable, MongoDB is indexed, and the test suite is hermetic and green. A formal read-only technical audit (2026-09-10) confirmed the system is **not fabricated** — the frontend shows no invented data — while identifying 15 findings across correctness, integrity, and capability. A remediation roadmap of 13 phases now closes those findings one commit at a time; the highest-severity correctness bug (a structurally unreachable alert subsystem) has already been fixed and verified.

> **The one-line takeaway**
>
> You have a working, honestly-evaluated deception pipeline with a solid deterministic core. The remaining value — and the **novelty** — lies in the "intelligence" layer: teaching the system to say *"I don't know / this is novel"*, to *predict the next move*, and to keep the LLM strictly advisory. Those three capabilities are what turn a good classifier into a publishable contribution.

2. What Has Been Built (current, verified state)

2.1 End-to-end pipeline

The verified data path (confirmed across **docker-compose.yml**, **log_collector.py**, **pipeline.py**, and **persistence_service.py**) is:

Stage	Component	What it does
1 Capture	Cowrie honeypot	Emulated SSH shell on loopback 127.0.0.1:2222; writes cowrie.json
2 Ingest	AsyncLogCollector	Binary, rotation- and truncation-safe tail of the JSON log
3 Normalise	EventNormalizer	JSON line -> NormalizedEvent with deterministic sha256 id
4 Features	FeatureExtractor / SessionAggregator	Incremental per-session behavioural features (tokenise only)
5 Classify	HybridClassifier	Rule -> calibrated ML -> optional Gemini, with recorded source
6 Score	RiskEngine	Additive, explainable 0–100 event risk; deterministic
7 Profile	SessionIntelligence	Cumulative session risk, stage reach, attacker aggregation
8 Persist	PersistenceService	MongoDB: events / sessions / attackers / alerts (indexed)
9 Serve	FastAPI + WebSockets	20 REST endpoints + live /ws/events and /ws/alerts
10 Present	React dashboard	7 analyst pages: dashboard, sessions, attackers, alerts, analytics

2.2 Core capabilities (present today)

- **Hybrid intent classification** — a three-source engine with an auditable precedence: a deterministic rule matcher, a calibrated LinearSVC ML core (TF-IDF word + char n-grams), and an *optional* Google Gemini 2.5 Flash layer for novel/ambiguous commands. Every event records which source decided it (RULE / ML / HYBRID / GEMINI / FALLBACK).
- **Deterministic risk scoring** — an additive 0–100 engine with severity mapping; the LLM can never move a score ("AI boundary enforcement").
- **MITRE ATT&CK mapping** — per-session tactic / technique attribution with evidence.
- **Attacker profiling** — persistent per-IP profiles with multi-session behavioural history.
- **Live streaming** — real-time event and (now working) alert broadcast over WebSockets.
- **Analytics** — intent distribution, severity breakdown, timelines, risk histograms, source heatmaps.
- **Privacy by design** — passwords redacted to [REDACTED] before persistence; credentials never broadcast.

2.3 Intent taxonomy (10 MITRE-aligned classes)

Intent	MITRE tactic	Example
RECONNAISSANCE	TA0043	uname -a, whoami
DISCOVERY	TA0007	ps aux, netstat -antp
CREDENTIAL_ACCESS	TA0006	cat /etc/shadow
EXECUTION	TA0002	wget .../payload.sh
PRIVILEGE_ESCALATION	TA0004	sudo su -
PERSISTENCE	TA0003	echo ... >> authorized_keys
EXPLOITATION	TA0002	() { :;}; echo VULN
EXFILTRATION	TA0010	tar czf loot.tar.gz /etc
CLEANUP	TA0005	rm -rf /var/log/auth.log
OTHER	—	login telemetry, benign input

2.4 Honest ML performance (both splits, as committed in metrics.json)

Evaluation split	Accuracy	Macro F1	Test N
Primary (same-template 80/20, de-duplicated)	94.49%	0.9229	272
Robustness (leave-families-out = true generalisation)	67.87%	0.5267	305
5-fold CV (LinearSVC)	85.39% ± 1.8%	—	—

Dataset: 1,360 unique commands across 242 template families, class-imbalanced (EXECUTION 318 ... PRIV_ESC 57). Weakest class RECONNAISSANCE (F1 0.714). The ~27-point gap between the two splits is reported openly, not hidden — it is precisely the room the hybrid rule + LLM layers exist to cover at runtime.

2.5 Quality gates

- **82 test functions** across 18 modules, fully hermetic (mongomock, mocked Gemini, tempfiles) — no live database or network needed. Strong exact-value assertions, not filler.
- **CI** (GitHub Actions): pytest on Python 3.13 plus an advisory pip-audit dependency scan.
- **Host safety verified**: no path exists by which attacker input executes on the host — emulated shell, loopback-only, read-only log mount, no exec/subprocess in the ingest path.

3. Remediation Progress (audit -> roadmap -> done)

A read-only audit produced 15 prioritised findings (F1–F15). The companion roadmap turns them into 13 phases. The table below is the live status; “done” items are backed by commits on the **remediation/int-aware** branch.

ID	Pri	Finding	Status
F4	P0	No git history; repo rooted at $HOME	DONE
—	P0	CI: hermetic pytest + advisory pip-audit (Phase 0)	DONE
F1	P0	Alert subsystem unreachable (event risk cap 65 vs 75 gate)	DONE
F5	P1	Gemini request timeout configured but never enforced	DONE
F6	P1	Unbounded in-memory growth of session / dedup caches	DONE
F7	P1	update_attacker recompute was O(sessions) per event	DONE
F2	P0	Optimistic accuracy framing (foreground honest split)	PENDING
F3	P0	Eval CSVs contradict prose; no latency percentiles	PENDING
F8	P1	No novelty/OOD/abstention; no predictive next-intent	PROPOSED
F9	P1	No auth/TLS; nginx headers; CORS too broad	PENDING
F10	P2	ML stack untested; adversarial/load offline-only	PENDING
F11	P2	Root containers; mutable tags (CVE scan added, advisory)	PARTIAL
F12	P2	Weak Gemini payload sanitisation; analyst lacks schema	PENDING
F13	P3	Misleading frontend fallbacks (null IP -> 127.0.0.1)	PENDING
F14	P3	Doc drift (Motor vs pymongo; Python 3.12 vs 3.13)	PENDING
F15	P3	GET /api/events/recent untested	PENDING

Committed on the remediation branch

Commit	What it delivered
472b99e	Baseline snapshot; repo re-rooted in the project (closes F4)
21d6ce2	CI workflow: hermetic pytest + advisory pip-audit (Phase 0)
382a724	Alerts now driven by session cumulative risk, not event risk (F1)
b37db46	Enforce configured Gemini request timeout (F5)
ed03575	Bound session + dedup caches to prevent unbounded growth (F6)
9d7e2f4	Attacker profile via server-side aggregation, O(1)-ish per event (F7)

Net effect: all P0 correctness (F1, F4) and all P1 reliability findings (F5–F7) are closed. What remains is scientific-integrity polish (F2, F3), security hardening (F9), test depth (F10), and — most importantly for novelty — the intelligence capabilities in F8.

4. Features You Can Add

Two tiers. Tier A is the enhancement backlog already scoped in your roadmap (Phases 4–12) — each ships with a measurable success criterion. Tier B is a set of higher-ambition ideas that go beyond the current roadmap and would most sharply raise the project's originality.

Tier A — On your existing roadmap (scoped, sequenced)

Phase	Feature	Why it matters
P4	Novelty / OOD / abstention	System can output KNOWN / AMBIGUOUS / LOW-CONF / NOVEL instead of forcing 1-of-10
P5	Predictive next-intent + sequence model	Forecast the next tactic; label OBSERVED / INFERRED / PREDICTED
P6	Decomposable risk v2 + alert priority	Itemised, summable risk contributions; ordered alert feed
P7	Behavioural attacker profiling + MITRE depth	Cluster attacker archetypes; tactic->technique->sub-technique
P8	Security hardening	API auth, WS origin/cap, tight CORS, nginx headers, non-root containers
P9	Adversarial-robustness CI gate	Promote unseen A/B/C/D set into a regression-gated metric
P10	ML-stack + regression tests	Load the real .joblib; fail CI if held-out accuracy drops
P11	Observability + honest benchmarks	Structured logs, per-stage latency p50/p95/p99 on documented HW
P12	Analyst dashboard + doc rewrite	Surface determinations, predictions, risk decomposition; fix fallbacks

Tier B — Beyond the roadmap (originality boosters)

These are my additions — not in your current roadmap. Each is tagged by how much it raises novelty. All can be built without violating the host-safety or no-fabrication rules.

- **Adaptive / intent-driven deception** **[HIGH novelty]**
  Use the *predicted* next intent to change what the emulated shell reveals in real time — e.g. dangle a fake credential file when CREDENTIAL_ACCESS is forecast, or a fake DB dump when EXFILTRATION looks imminent. This closes the loop from passive profiling to active engagement.
- **Behavioural attacker fingerprinting (cross-IP re-identification)** **[HIGH novelty]**
  Command-style “stylometry” — timing rhythm, typo/retry patterns, tool signatures, command ordering — to link sessions from different source IPs to a single actor. Turns per-IP profiles into per-*actor* profiles.
- **Active-learning feedback loop** **[MED-HIGH novelty]**
  Let an analyst confirm or correct a determination in the dashboard; corrections feed a retraining queue and a drift monitor. Directly attacks the 67.87% generalisation gap over time.
- **Campaign graph correlation** **[MED-HIGH novelty]**
  Build a graph over attackers / sessions / techniques / payload hashes to surface coordinated campaigns and shared infrastructure, not just isolated sessions.
- **Calibration & drift dashboard** **[MED novelty]**
  Reliability diagrams, Expected Calibration Error over time, per-class drift — a natural, on-brand extension of the project's scientific-integrity theme, and it feeds the abstention thresholds.
- **LLM-authored SOC narrative (guarded)** **[MED novelty]**
  One-click natural-language session report + auto MITRE writeup, under the same non-authoritative, schema-validated, never-executed guardrails already used for classification.

- **Threat-intel enrichment** **[MED (integration)]**
  Read-only, cached lookups (GreyNoise / AbuseIPDB / Tor-exit) to add reputation context to each source IP. Enriches profiles without touching the deterministic risk authority.
- **Interop exports: STIX/TAXII, MISP, SIEM/SOAR webhooks** **[MED (integration)]**
  Emit findings to Slack / Splunk / Elastic and export indicators in standard formats — makes the platform deployable inside a real SOC.

5. Novelty — What Actually Makes This Publishable

Honest starting point: honeypots (Cowrie), supervised command-intent classifiers, LLM security assistants, and MITRE mapping all already exist *individually*. So the novelty of INT-AWARE is **not** any one of those parts. It is (a) the **trust architecture** that binds them and (b) two capabilities that most deployed classifiers simply do not have. Below, each pillar is marked with whether it exists today or is on the roadmap — so the contribution can be stated without over-claiming.

> **The contribution, in one sentence**
> *An intent-aware honeypot-analytics pipeline in which a probabilistic ML core and an optional LLM operate strictly beneath a deterministic, explainable authority — extended so the system can flag NOVEL / unknown attacker behaviour (abstention) and forecast the next tactic with explicit epistemic provenance (OBSERVED / INFERRED / PREDICTED).*

Pillar 1 — A deterministic-authority trust boundary for LLMs in security

Status: IMPLEMENTED today.

The Gemini layer is consulted *only* for commands where both the rule engine is weak and the ML core is not confident. Its output is schema-constrained, temperature-pinned, validated, and **never executed**. Crucially, it can never change a risk score — all risk, severity, and alerting are computed by a deterministic additive engine, and the whole system runs fully with the LLM switched off (graceful degradation). Most LLM-in-security work either lets the model drive decisions (opaque, prompt-injectable, irreproducible) or bolts it on as an unchecked black box. Demonstrating a **reproducible, falsifiable** pipeline where the LLM is provably subordinate to a glass-box core is a genuine trustworthy-AI contribution, not just plumbing.

Pillar 2 — Intent-awareness WITH abstention and novelty detection

Status: ROADMAP (Phase 4). Designed, dataset ready, not yet built.

Today the classifier always returns one of 10 classes — it cannot say “I don't know.” The proposed step: emit KNOWN / AMBIGUOUS / LOW-CONFIDENCE / NOVEL(OOD) using calibrated max-probability, top-2 margin, and an out-of-distribution score (distance to training centroids or a one-class model). Abstention and OOD detection on *honeypot command streams* is under-explored, and the unseen A/B/C/D evaluation set to validate it already exists in the repo. This is the single highest-leverage novelty item: “intent-**aware**” honestly requires the system to know the limits of what it knows.

Pillar 3 — Predictive, provenance-labelled attack-stage intelligence

Status: ROADMAP (Phase 5).

Move from “what happened” to “what is likely next”: a transition model over observed intent sequences forecasts the next tactic with a calibrated probability. The distinctive twist is **epistemic provenance labelling** — every piece of intelligence is tagged OBSERVED (it happened), INFERRED (deduced from current state), or PREDICTED (a forecast) — so a prediction is never rendered as fact. That honesty-by-construction is itself a defensible design contribution and pairs naturally with adaptive deception (Tier B).

Pillar 4 — Audit-driven, dual-split honest evaluation (methodological)

Status: IMPLEMENTED / ongoing.

Not research-novelty per se, but a real differentiator at this tier and a strengthener for any paper: both same-template (94.49%) and leave-families-out (67.87%) accuracy are reported, no metric is fabricated, every figure is regenerated from a seeded script, and a formal read-only audit drives a finding-by-finding remediation. Reviewers reward this; it is rare in student-tier security projects.

How INT-AWARE sits against the alternatives

Approach	Detects intent	Says novel/unknown	Predicts next step	LLM used safely	Explainable risk
Raw Cowrie honeypot	No	—	No	—	No
Supervised command classifier	Yes (forced)	No	No	—	No
LLM-only SOC assistant	Yes	Sometimes	Sometimes	No (LLM decides)	No (opaque)
INT-AWARE — today	Yes	Not yet	Not yet	Yes	Yes
INT-AWARE — with P4–P6	Yes	Yes	Yes	Yes	Yes (decomposable)

The last two rows are the pitch: the current system already occupies a defensible corner (safe LLM + explainable risk), and finishing Phases 4–6 lands the full “intent-aware, abstaining, predictive, explainable” claim — which no single row above matches.

Candidate framings for a paper / viva

- “Knowing what you don't know: abstention and novelty detection for honeypot command-intent classification.”
- “Advisory, not authoritative: a deterministic-core trust boundary for LLMs in cyber-deception.”
- “From detection to forecast: provenance-labelled next-intent prediction in an intent-aware honeypot.”

6. Recommended Next Steps (prioritised)

#	Do this	Because	Effort
1	Phase 2: foreground the honest metric; reconcile eval CSVs vs prose	Closes the last P0 integrity gap; protects credibility before adding features	S
2	Phase 4: novelty / OOD / abstention	The single biggest novelty lever; dataset already exists	M
3	Phase 5: predictive next-intent with provenance labels	Turns the classifier into forecasting; strong demo/viva moment	M
4	Phase 10: load the real .joblib in tests + regression gate	The ML stack is currently untested; protects the model claim	S-M
5	Phase 8: baseline auth + nginx headers + non-root containers	Required before any non-loopback demo; low-risk hardening	M
6	Tier B: pick ONE originality booster (adaptive deception or fingerprinting)	One well-executed Tier-B feature is what makes the project memorable	M-L

Effort key: S = small (hours), M = medium (days), L = larger (1–2 weeks).

Every figure in this report is quoted from committed artifacts (PROJECT_AUDIT.md, ROADMAP.md, docs/evaluation/*, metrics.json) or the git history of remediation/int-aware, in keeping with the project's own no-fabrication rule. Items marked ROADMAP or PROPOSED are not yet implemented and are labelled as such throughout.